

```

[Unit]
Description=Docker Application Container Engine
Documentation=https://docs.docker.com
After=network-online.target docker.socket firewalld.service
Wants=network-online.target
Requires=docker.socket

[Service]
Type=notify
# the default is not to use systemd for cgroups because the delegate issues a
# exists and systemd currently does not support the cgroup feature set requir
# for containers run by docker
ExecStart=/usr/bin/dockerd -H fd://
ExecReload=/bin/kill -s HUP $MAINPID
LimitNOFILE=1048576
# Having non-zero Limit*s causes performance problems due to accounting overh
# in the kernel. We recommend using cgroups to do container-local accounting.
LimitNPROC=infinity
LimitCORE=infinity
# Uncomment TasksMax if your systemd version supports it.
# Only systemd 226 and above support this version.
TasksMax=infinity
TimeoutStartSec=0
# set delegate yes so that systemd does not reset the cgroups of docker conta
Delegate=yes
# kill only the docker process, not all processes in the cgroup
KillMode=process
# restart the docker process if it exits prematurely
Restart=on-failure
StartLimitBurst=3
StartLimitInterval=60s
EnvironmentFile=/etc/default/docker

[Install]
WantedBy=multi-user.target

```

Figure 3: Modifying docker configuration

Then verify again with the `echo` command to ensure that the environment variable is updated properly.

- c. Repeat the above steps for all environment variables like `https_proxy`, `HTTP_PROXY` and `HTTPS_PROXY`.
- 2) **Problem:** Sometimes the proxy will not persist across environments and configurations.

If the `config.json` file is updated with the following proxy parameters, this problem will be overcome. Figure 2 shows how to edit this file and set up the parameters.

- a. Update the `~/.docker/config.json` file.
- b. Further to the above step, please update the daemon file `/etc/default/docker` with all environment variables.
- c. Now add the reference of the `/etc/default/docker` file as a variable in the file `/lib/systemd/system/docker.service` as shown in Figure 3.
- d. Restart the Docker services, as follows:

```

# ubuntu@test-vm1:~$ sudo systemctl daemon-reload
# ubuntu@test-vm1:~$ sudo systemctl restart docker
# ubuntu@test-vm1:~$ sudo systemctl show --property
Environment docker

```

How to override proxy configuration (during runtime)

This method helps in building Docker container images and further helps to override the host proxy to avoid future problems related to the proxy setup.

This procedure helps to override the proxy settings made while building the Dockerfile without modifying the contents in `/etc/default/docker` configuration. The Docker build provides an argument called `build-arg`, which can be utilised in this case. These values will persist in the respective image builds. If the same image is used, the proxy setup will be by default available as it was set up.

Note: End users who would like to connect directly to the Internet without a proxy, may not need this setup.

The following set of commands and configurations are used to set up this proxy:

```

$ sudo docker build --build-arg \ http_proxy="http://
username:password@proxy:port/" --build-arg \ https_
proxy="https:// username:password@proxy:port /"
--network=host.

```

- b. We can use the `--build-arg` argument without the values as well. These values can be populated from the environment variables like `http_proxy`, as shown below:

```

$ export http_proxy=http://proxyaddress:port

```

```

$ docker build --build-arg http_proxy .

```

We hope that this article will help those who face proxy issues while configuring Docker containers. We have tried to address most of the proxy configuration issues with this step-by-step approach, giving the required configuration details, commands and relevant screenshots. This article may not be of use if a direct Internet connection is used without a proxy. **END** 🐧

References

- [1] <https://www.zdnet.com/article/container-adoption-speeds-up-to-the-detriment-of-vms/>
- [2] <https://www.varonis.com/blog/what-is-a-proxy-server/>
- [3] <https://docs.docker.com/engine/reference/commandline/build/#set-build-time-variables---build-arg>
- [4] <https://docs.docker.com/engine/reference/commandline/build/#set-build-time-variables---build-arg>
- [5] <https://docs.docker.com/>
- [6] <https://hub.docker.com/>
- [7] <https://www.varonis.com/blog/what-is-a-proxy-server/>

By: Srikanth Mohan, Ganesan Nagasamy and Shashidhar Soppin

Srikanth Mohan is a senior architect with 15+ years of experience in IT. He specialises in test automation and performance testing. You can contact him at tmsrikanth1982@gmail.com.

Ganesan Nagasamy has 18+ years of experience in the software industry. He specialises in test automation, non-functional testing, the cloud, and agile technology. He is a certified Scrum product owner. You can contact him at ganesan.nagasamy@gmail.com.

Shashidhar Soppin is a senior architect with 17+ years of experience in the IT industry. He specialises in virtualisation, Docker, the cloud, AI-ML, deep learning and OpenStack. He has many patents and published papers to his credit. He can be contacted at shashi.soppin@gmail.com.